

LINQ to Entities in Entity Framework Core

In this article, I will discuss **LINQ to Entities in Entity Framework Core (EF Core)**. Please read our previous article discussing [Entity States in Entity Framework Core](#). We will work with the same example we have worked on so far.

What is LINQ?

LINQ (Language Integrated Query) is a powerful query language introduced by Microsoft that allows developers to query collections of data in a more readable and concise manner. LINQ provides a unified approach to query different types of data sources, such as collections, XML, databases, etc. LINQ can be used with various types of data sources:

- **LINQ to Objects:** This is used to query in-memory collections like arrays and lists.
- **LINQ to SQL:** This is used to query SQL Server databases.
- **LINQ to Entities:** This is used to query databases via Entity Framework.
- **LINQ to XML:** This is used to query XML documents.

LINQ queries can be written in two syntaxes:

- **Query Syntax:** Similar to SQL syntax but embedded in C#.
- **Method Syntax:** Uses extension methods and lambda expressions to perform operations on collections.

What is LINQ-to-Entities in Entity Framework Core?

LINQ-to-Entities is a subset of LINQ (Language Integrated Query) that works with Entity Framework Core to query and interact with database data. It allows developers to write strongly typed queries against the Entity Framework Core data model using C# syntax. These queries are translated into SQL queries that are executed against the database, and the results are returned as objects of the entity types defined in the model.

When you perform operations such as filtering, sorting, grouping, or joining using LINQ, the DbSet converts these operations into SQL queries, executes them on the database, and then maps the results back to entity objects in your application. Since the queries are strongly typed, they provide compile-time checking and IntelliSense support and are easier to maintain.

What is Projection in LINQ?

Projection in LINQ refers to the process of transforming the data returned by a query into a different shape or structure. This is typically done using the select keyword in query syntax or the Select method in method syntax. Projection allows us to specify which properties or fields we want to include in the result and how they should be organized.

Examples to Understand LINQ-to-Entities in Entity Framework Core:

We will work with the same application we have been working on so far. If you are coming directly to this article, please be ready with the following model and DbContext classes. In our application, we are using the following Student and Branch Entities.

Student.cs

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
```

```

        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public DateTime? DateOfBirth { get; set; }
        public string? Gender { get; set; }
        public string? Email { get; set; }
        public string? PhoneNumber { get; set; }
        public DateTime EnrollmentDate { get; set; }
        // Navigation property representing the Branch the student is
        enrolled in
        public virtual Branch? Branch { get; set; }
    }
}

```

Branch.cs

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Branch
    {
        public int BranchId { get; set; }
        public string? BranchName { get; set; }
        public string? Description { get; set; }
        public string? PhoneNumber { get; set; }
        public string? Email { get; set; }
        // Collection navigation property representing the students enrolled
        in the branch
        public ICollection<Student>? Students { get; set; }
    }
}

```

EFCoreDbContext Class:

The following is our DbContext class.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    // EFCoreDbContext is your custom DbContext class that extends the base DbContext
    by EF Core.
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Disabling the EF Core Log
            // Display the generated SQL queries in the Console window
            // optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            // Configure the connection string
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB1;Trusted_Connection=True;TrustServerCertificate=True");
        }
    }
}

```

```

    }
    // DbSet<Student> corresponds to the Students table in the database.
    // It allows EF Core to track and manage Student entities.
    public DbSet<Student> Students { get; set; }
    // DbSet<Branch> corresponds to the Branches table in the database.
    // It allows EF Core to track and manage Branch entities.
    public DbSet<Branch> Branches { get; set; }
}
}

```

Deleting the OLD data from the Database Tables

Before proceeding further, let us first delete all the old data by executing the following SQL script in SQL Server. We cannot apply the TRUNCATE command to a primary key table that is referenced by a foreign key constraint. Instead, we first truncate the foreign key table (Students) to remove all rows. Then, we delete all records from the primary key table (Branches) because DELETE allows us to remove records even when there is a foreign key constraint. After that, we reseed the identity column of the Branches table.

```

-- Truncate the Foreign Key Table
TRUNCATE TABLE Students;
GO
-- Delete All Records from the Primary Key Table
DELETE FROM Branches;
GO
-- RESEED The Identity
DBCC CHECKIDENT ('EFCoreDB1.dbo.Branches', RESEED, 0);
GO

```

SQL Script Explanation:

- **Truncating the Foreign Key Table (Students):** This step removes all rows from the Students table and resets its identity column (if it has one) because TRUNCATE automatically resets the identity seed to the defined start value.
- **Deleting Records from the Primary Key Table (Branches):** Since TRUNCATE cannot be applied to a table that is referenced by a foreign key, we use DELETE to remove all records from the Branches table. However, DELETE does not reset the identity seed by default.
- **Reseeding the Identity Column:** The DBCC CHECKIDENT command resets the identity column in the Branches table to the specified seed value (0 in this case). This ensures that the next record inserted will start with an identity value of 1.

Inserting Dummy Data to Understand EF Core with LINQ

We will work with the following data within the Branches table.

```
SELECT * FROM [Branches]
```

BranchId	BranchName	Description	PhoneNumber	Email
1	Computer Science Engineering	Department focused on softwa...	555-1010	cse@dotnettutorials.net
2	Mechanical Engineering	Department focused on design...	555-2020	me@dotnettutorials.net
3	Electrical Engineering	Department focused on electri...	555-3030	ee@dotnettutorials.net
4	Civil Engineering	Department focused on infrastr...	555-4040	ce@dotnettutorials.net

We will work with the following data within the Students table.

```
SELECT * FROM [Students]
```

StudentId	FirstName	LastName	DateOfBirth	Gender	Email	PhoneNumber	EnrollmentDate	BranchId
1	Alice	Wong	2001-02-14...	Female	alice.wong@dotnettutorials.net	555-1111	2023-08-15 ...	1
2	Bob	Johnson	2002-06-22...	Male	bob.johnson@dotnettutorials.net	555-2222	2023-08-16 ...	2
3	Carol	Martinez	2000-11-02...	Female	carol.martinez@dotnettutorials.net	555-3333	2023-08-17 ...	3
4	David	Kim	1999-12-19...	Male	david.kim@dotnettutorials.net	555-4444	2023-08-18 ...	4
5	Eve	Nguyen	2001-05-03...	Female	eve.nguyen@dotnettutorials.net	555-5555	2023-08-19 ...	1
6	Frank	Connor	2002-09-11...	Male	frank.oconnor@dotnettutorials.net	555-6666	2023-08-20 ...	2
7	Grace	Lee	2003-01-25...	Female	grace.lee@dotnettutorials.net	555-7777	2023-08-21 ...	3
8	Henry	Patel	2001-07-08...	Male	henry.patel@dotnettutorials.net	555-8888	2023-08-22 ...	4
9	Ivy	Zhang	2002-03-15...	Female	ivy.zhang@dotnettutorials.net	555-9999	2023-08-23 ...	1
10	Jack	Wilson	2000-10-29...	Male	jack.wilson@dotnettutorials.net	555-0000	2023-08-24 ...	2

Please use the below SQL Scripts to insert the above into the database tables.

```
-- Insert Data into Branches Table
USE EFCoreDB1
GO
INSERT INTO Branches (BranchName, Description, PhoneNumber, Email)
VALUES
('Computer Science Engineering', 'Department focused on software development,
algorithms, and computer systems.', '555-1010', 'cse@dotnettutorials.net'),
('Mechanical Engineering', 'Department focused on design, analysis, and
manufacturing of mechanical systems.', '555-2020', 'me@dotnettutorials.net'),
('Electrical Engineering', 'Department focused on electrical systems,
electronics, and signal processing.', '555-3030', 'ee@dotnettutorials.net'),
('Civil Engineering', 'Department focused on infrastructure, environmental,
and construction engineering.', '555-4040', 'ce@dotnettutorials.net');
GO
-- Insert Data into Students Table
INSERT INTO [dbo].[Students] (FirstName, LastName, DateOfBirth, Gender,
Email, PhoneNumber, EnrollmentDate, BranchId)
VALUES
('Alice', 'Wong', '2001-02-14', 'Female', 'alice.wong@dotnettutorials.net',
'555-1111', '2023-08-15', 1),
```

```
( 'Bob', 'Johnson', '2002-06-22', 'Male', 'bob.johnson@dotnettutorials.net',
'555-2222', '2023-08-16', 2),
( 'Carol', 'Martinez', '2000-11-02', 'Female',
'carol.martinez@dotnettutorials.net', '555-3333', '2023-08-17', 3),
( 'David', 'Kim', '1999-12-19', 'Male', 'david.kim@dotnettutorials.net', '555-
4444', '2023-08-18', 4),
( 'Eve', 'Nguyen', '2001-05-03', 'Female', 'eve.nguyen@dotnettutorials.net',
'555-5555', '2023-08-19', 1),
( 'Frank', 'Connor', '2002-09-11', 'Male',
'frank.oconnor@dotnettutorials.net', '555-6666', '2023-08-20', 2),
( 'Grace', 'Lee', '2003-01-25', 'Female', 'grace.lee@dotnettutorials.net',
'555-7777', '2023-08-21', 3),
( 'Henry', 'Patel', '2001-07-08', 'Male', 'henry.patel@dotnettutorials.net',
'555-8888', '2023-08-22', 4),
( 'Ivy', 'Zhang', '2002-03-15', 'Female', 'ivy.zhang@dotnettutorials.net',
'555-9999', '2023-08-23', 1),
( 'Jack', 'Wilson', '2000-10-29', 'Male', 'jack.wilson@dotnettutorials.net',
'555-0000', '2023-08-24', 2);
GO
```

Let us proceed and see how we can use LINQ to Entities Queries to perform different types of Operations, such as Searching, Filtering, Sorting, Grouping, and Joining.

How Do We Implement Searching Using LINQ to Entities in Entity Framework Core?

Searching in the context of databases involves querying the database to find specific records that match a given search criterion. For example, if we want to find a student with a specific FirstName or search for all students in a particular branch, we would perform a search operation. Searching can be implemented using both LINQ Query Syntax and LINQ Method Syntax in C#.

Let us see how to implement searching using both LINQ Query Syntax and LINQ Method Syntax. So, please modify the Program class code as follows. The following code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Initialize the DbContext
                using (var context = new EFCoreDbContext())
                {
                    // Define the search criteria (searching for a student
                    with the first name "Alice")
                    string searchFirstName = "Alice";

                    // LINQ Query Syntax to search for a student by first
                    name
```


Let us understand how to implement filtering using both LINQ Query Syntax and LINQ Method Syntax in our Application. To better understand this, please modify the Program class as follows: Here, we filter students based on multiple criteria, such as BranchId and Gender. The following code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Initialize the DbContext
                using (var context = new EFCoreDbContext())
                {
                    // Define the filtering criteria
                    string branchName = "Computer Science Engineering"; //
Branch name filter
                    string gender = "Female"; // Gender filter

                    // LINQ Query Syntax to filter students by branch name
and gender with eager loading
                    var filteredStudentsQS = (from student in
context.Students
                                            .Include(s => s.Branch) // Eager
loading of the Branch property
                                            where student.Branch.BranchName
== branchName && student.Gender == gender
                                            select student).ToList();

                    // LINQ Method Syntax to filter students by branch name
and gender with eager loading
                    var filteredStudents = context.Students
                                            .Include(s => s.Branch) //
Eager loading of the Branch property
                                            .Where(s =>
s.Branch.BranchName == branchName && s.Gender == gender)
                                            .ToList();

                    // Check if any students match the filtering criteria
                    if (filteredStudentsQS.Any())
                    {
                        // Iterate through the filtered students and display
their details
                        foreach (var student in filteredStudentsQS)
                        {
                            Console.WriteLine($"Student Found:
{student.FirstName} {student.LastName}, Branch: {student.Branch.BranchName},
```

```

Gender: {student.Gender}");
        }
    }
    else
    {
        // Output if no students match the filtering criteria
        Console.WriteLine("No students found matching the
given criteria.");
    }
}
}
}
catch (Exception ex)
{
    // Exception handling: log the exception message
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}

```

Output:

```

Student Found: Alice Wong, Branch: Computer Science Engineering, Gender: Female
Student Found: Eve Nguyen, Branch: Computer Science Engineering, Gender: Female
Student Found: Ivy Zhang, Branch: Computer Science Engineering, Gender: Female

```

How Do We Implement Sorting Using LINQ to Entities in Entity Framework Core?

Sorting is the process of arranging data in a particular order, either ascending (smallest to largest) or descending (largest to smallest). In a database context, sorting is often used to organize query results based on one or more columns. For example, you might want to sort students by their LastName alphabetically or by their EnrollmentDate to see the most recently enrolled students first.

Let us see how to implement sorting using both LINQ Query Syntax and Method Syntax in our Application. We will sort students by their Gender in ascending order and by their EnrollmentDate in descending order. Please modify the Program class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Sorting students by Gender ascending and
                    EnrollmentDate descending using Query Syntax

```

```

        var sortedStudentsQuerySyntax = (from student in
context.Students
                                         orderby student.Gender
ascending, student.EnrollmentDate descending
                                         select student).ToList();
        // Sorting students by LastName ascending and
EnrollmentDate descending using Method Syntax
        var sortedStudentsMethodSyntax = context.Students
                                         .OrderBy(s =>
s.Gender) // Primary sort by Gender in ascending order

        .ThenByDescending(s => s.EnrollmentDate) // Secondary sort by EnrollmentDate
in descending order
                                         .ToList();

        // Check if any students are found
        if (sortedStudentsQuerySyntax.Any())
        {
            // Iterate through the sorted students and display
their details
            foreach (var student in sortedStudentsQuerySyntax)
            {
                // Output the student's details including Gender
and enrollment date
                Console.WriteLine($"Student: {student.LastName}
{student.FirstName}, Gender: {student.Gender}, Enrollment Date:
{student.EnrollmentDate.ToShortDateString()}");
            }
        }
        else
        {
            // Output if no students are found
            Console.WriteLine("No students found.");
        }
    }
}
catch (Exception ex)
{
    // Exception handling: log the exception message
    // This catches any errors that occur during database access
or LINQ operations
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

When you run the program, the output will show students sorted by Gender in ascending order and then sorted by enrollment date in descending order within that Gender. The output might look like the one below:

```

Student: Zhang Ivy, Gender: Female, Enrollment Date: 23-08-2023
Student: Lee Grace, Gender: Female, Enrollment Date: 21-08-2023
Student: Nguyen Eve, Gender: Female, Enrollment Date: 19-08-2023
Student: Martinez Carol, Gender: Female, Enrollment Date: 17-08-2023
Student: Wong Alice, Gender: Female, Enrollment Date: 15-08-2023
Student: Wilson Jack, Gender: Male, Enrollment Date: 24-08-2023
Student: Patel Henry, Gender: Male, Enrollment Date: 22-08-2023
Student: Connor Frank, Gender: Male, Enrollment Date: 20-08-2023
Student: Kim David, Gender: Male, Enrollment Date: 18-08-2023
Student: Johnson Bob, Gender: Male, Enrollment Date: 16-08-2023

```

How Do We Implement Grouping Using LINQ to Entities in Entity Framework Core?

Grouping is the process of organizing data into groups based on a specified key. In the context of databases, grouping allows us to categorize records that share a common attribute. This is particularly useful for performing aggregate operations (such as counting, summing, or averaging) on each group.

Let us see how to implement grouping using LINQ Query Syntax and LINQ Method Syntax. We will group students by their branch and count the number of students in each branch. Please modify the Program class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Grouping students by their Branch using Query Syntax
                    var groupedStudentsQuerySyntax = (from student in
context.Students
                                                    .Include(s => s.Branch)
// Eager loading of the Branch property
group student by
student.Branch.BranchName into studentGroup //Group Students by BranchName
into studentGroup
select new
{
    // studentGroup.Key
    BranchName =
studentGroup.Key,
// Count the number
of students in each group

```

```

StudentCount =
studentGroup.Count()
    }).ToList();
    // Grouping students by their Branch using Method Syntax
    //var groupedStudentsMethodSyntax = context.Students
    //                                     .Include(s =>
s.Branch) // Eager loading of the Branch property
    //                                     .GroupBy(s =>
s.Branch.BranchName) // Group students by BranchName
    //                                     .Select(g =>
new
    //                                     {
    //                                     // g.Key
is the BranchName in this case
    //                                     BranchName
= g.Key,
    //                                     // Count
the number of students in each group
    //                                     }
StudentCount = g.Count()
    //                                     })
    //                                     .ToList();
    // Check if any groups are found
    if (groupedStudentsQuerySyntax.Any())
    {
        // Iterate through the grouped students and display
their details
        foreach (var group in groupedStudentsQuerySyntax)
        {
            // Output the Branch name and the number of
students in that branch
            Console.WriteLine($"Branch: {group.BranchName},
Number of Students: {group.StudentCount}");
        }
    }
    else
    {
        // Output if no students are found
        Console.WriteLine("No students found.");
    }
}
}
catch (Exception ex)
{
    // Exception handling: log the exception message
    // This catches any errors that occur during database access
or LINQ operations
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}

```

```
}
}
```

When you run the above code, you will get the following output:

```
Branch: Civil Engineering, Number of Students: 2
Branch: Computer Science Engineering, Number of Students: 3
Branch: Electrical Engineering, Number of Students: 2
Branch: Mechanical Engineering, Number of Students: 3
```

Now, we will group students by their branch, count the number of students in each branch, and display the list of each student's details within that branch. Please modify the Program class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Grouping students by their Branch using Query Syntax
                    var groupedStudentsQuerySyntax = (from student in
context.Students
                                                    .Include(s => s.Branch)
// Eager loading of the Branch property
group student by
student.Branch.BranchName into studentGroup
select new
{
    // studentGroup.Key
    BranchName =

// Count the number
StudentCount =

// Retrieve the list
of students in each group
```

```

studentGroup.ToList()

Students =
    }).ToList();

    // Grouping students by their Branch using Method Syntax
    //var groupedStudentsMethodSyntax = context.Students
    //                                     .Include(s =>
s.Branch) // Eager loading of the Branch property
    //                                     .GroupBy(s =>
s.Branch.BranchName) // Group students by BranchName
    //                                     .Select(g =>
new
    //                                     {
    //                                     // g.Key
    //                                     BranchName
    //                                     = g.Key,
    //                                     // Count
    //                                     the number of students in each group
    //                                     StudentCount = g.Count(),
    //                                     //
    //                                     Retrieve the list of students in each group
    //                                     //
    //                                     g.ToList()
    //                                     //
    //                                     //
    //                                     Students =
    //                                     })
    //                                     .ToList();

    // Check if any groups are found
    if (groupedStudentsQuerySyntax.Any())
    {
        // Iterate through the grouped students and display
        // their details
        foreach (var group in groupedStudentsQuerySyntax)
        {
            // Output the Branch name and the number of
            // students in that branch
            Console.WriteLine($"\\nBranch: {group.BranchName},
            Number of Students: {group.StudentCount}");

            // Display details of each student in the branch
            foreach (var student in group.Students)
            {
                Console.WriteLine($"\\tStudent:
                {student.FirstName} {student.LastName}, Email: {student.Email}, Enrollment
                Date: {student.EnrollmentDate.ToShortDateString()}");
            }
        }
    }
}

```

```

        else
        {
            // Output if no students are found
            Console.WriteLine("No students found.");
        }
    }
}
catch (Exception ex)
{
    // Exception handling: log the exception message
    // This catches any errors that occur during database access
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

Now, run the application. The output will show students grouped by their Branch, the number of students in each branch, and detailed information about each student, as shown in the image below:

```

Branch: Civil Engineering, Number of Students: 2
Student: David Kim, Email: david.kim@dotnettutorials.net, Enrollment Date: 18-08-2023
Student: Henry Patel, Email: henry.patel@dotnettutorials.net, Enrollment Date: 22-08-2023

Branch: Computer Science Engineering, Number of Students: 3
Student: Alice Wong, Email: alice.wong@dotnettutorials.net, Enrollment Date: 15-08-2023
Student: Eve Nguyen, Email: eve.nguyen@dotnettutorials.net, Enrollment Date: 19-08-2023
Student: Ivy Zhang, Email: ivy.zhang@dotnettutorials.net, Enrollment Date: 23-08-2023

Branch: Electrical Engineering, Number of Students: 2
Student: Carol Martinez, Email: carol.martinez@dotnettutorials.net, Enrollment Date: 17-08-2023
Student: Grace Lee, Email: grace.lee@dotnettutorials.net, Enrollment Date: 21-08-2023

Branch: Mechanical Engineering, Number of Students: 3
Student: Bob Johnson, Email: bob.johnson@dotnettutorials.net, Enrollment Date: 16-08-2023
Student: Frank Connor, Email: frank.oconnor@dotnettutorials.net, Enrollment Date: 20-08-2023
Student: Jack Wilson, Email: jack.wilson@dotnettutorials.net, Enrollment Date: 24-08-2023

```

How Do We Implement Joining Using LINQ to Entities in Entity Framework Core?

Joining in LINQ allows us to combine data from two or more sources (usually tables) based on a related column between them. In a relational database, joins are used to retrieve data that is spread across multiple tables. Joining is a fundamental operation in database querying, allowing us to bring together related data from different entities.

Let us understand how to implement joining using both LINQ Query Syntax and LINQ Method Syntax in our Console Application. We will Join the Students table with the Branches table to display each student's full name, email, enrollment date, and the branch they are enrolled in. Please modify the Program class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

{
    try
    {
        using (var context = new EFCoreDbContext())
        {
            // Joining Students and Branches using Query Syntax (LINQ
query)
            var studentsWithBranchesQuerySyntax = (from student in
context.Students // Loop over the Students table
            join branch in
context.Branches // Perform an inner join with the Branches table
            on
student.Branch.BranchId equals branch.BranchId // Define the join condition
based on BranchId
            select new //
Create an anonymous object containing selected fields from both tables
            {

student.FirstName, // Select the student's first name
student.LastName, // Select the student's last name
            student.Email,
// Select the student's email
student.EnrollmentDate, // Select the student's enrollment date
branch.BranchName // Select the corresponding branch name
            }).ToList(); //
Execute the query and convert the result to a list
            // Joining Students and Branches using Method Syntax
(LINQ method chaining)
            //var studentsWithBranchesMethodSyntax = context.Students
// Start with the Students table
            //
            .Join(context.Branches, // Join with the Branches table
            //
student => student.Branch.BranchId, // Define the key from the Students table
for the join (BranchId)
            //
branch => branch.BranchId, // Define the key from the Branches table for the
join (BranchId)
            //
(student, branch) => new // Create an anonymous object for each joined record
            {

student.FirstName, // Select the student's first name
            //
student.LastName, // Select the student's last name
            //
student.Email, // Select the student's email

```


- The branch name.
- The number of students in the branch.
- The average enrollment date of students in that branch.
- A list of students in that branch, sorted by their last name.

Please modify the Program class as follows to implement the above example. The following example code is self-explained, so please read the comment lines for a better understanding.

[illegible]

```

{
    BranchName =
branch.BranchName, // The name of the branch
    StudentCount =
branch.Students.Count(), // Count the number of students in the branch
    // Calculate the
average enrollment date (as ticks) of the students in the branch
    AverageEnrollmentDate
branch.Students.Average(s => s.EnrollmentDate.Ticks),
    // Sort students by
LastName ascending, then by FirstName ascending
    Students =
branch.Students.OrderBy(s => s.LastName).ThenBy(s => s.FirstName).ToList()
});
    // LINQ Method Syntax:
    // This query achieves the same goal as the above LINQ
Query Syntax but using method syntax.
    var branchDetailsMethodSyntax = context.Branches
    // Join the Branches
and Students tables on BranchId
    .Join(context.Students,
    branch =>
branch.BranchId,
    student =>
student.Branch.BranchId,
    (branch,
student) => new { branch, student })
    // Group the joined
data by BranchId and BranchName
    .GroupBy(bs => new
{ bs.branch.BranchId, bs.branch.BranchName })
    .AsEnumerable() //
Switch to client-side evaluation for further processing
    .Select(g => new
{
    BranchName =
g.Key.BranchName, // The name of the branch
    StudentCount =
g.Count(), // Count the number of students in the branch
    // Calculate the
average enrollment date (as ticks) of the students in the branch
    AverageEnrollmentDate = g.Average(bs => bs.student.EnrollmentDate.Ticks),
    // Sort student
by LastName ascending, then by FirstName ascending
    Students =
g.Select(bs => bs.student).OrderBy(s => s.LastName).ThenBy(s =>
s.FirstName).ToList()
    })
}

```

```

                                                                    .ToList(); //
Convert the result to a list for further processing
    // Display the results:
    // Check if there are any branches with students to display
    if (branchDetailsQuerySyntax.Any())
    {
        // Iterate over each branch in the query result
        foreach (var branch in branchDetailsQuerySyntax)
        {
            // Display the branch name
            Console.WriteLine($"\\nBranch:
{branch.BranchName}");

            // Display the number of students in the branch
            Console.WriteLine($"Number of Students:
{branch.StudentCount}");

            // Convert the average enrollment date (in ticks)
            to a DateTime and display it
            Console.WriteLine($"Average Enrollment Date: {new
DateTime(Convert.ToInt64(branch.AverageEnrollmentDate)).ToShortDateString()}");
            // Display details of each student in the branch
            foreach (var student in branch.Students)
            {
                Console.WriteLine($"    Student:
{student.LastName}, {student.FirstName} - Enrollment Date:
{student.EnrollmentDate.ToShortDateString()}, Email: {student.Email}");
            }
        }
    }
    else
    {
        // If no branch details are found, display a message
        indicating so
        Console.WriteLine("No branch details found.");
    }
}
}
catch (Exception ex)
{
    // Exception handling: log the exception message if something
    goes wrong during the process
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

When you run the program, the output will show detailed information for each branch, including the branch name, the number of students in that branch, the average enrollment date of the students, and a list of students sorted by last name, as shown in the below image:

=====Branch Wise Report=====

Branch: Computer Science Engineering

Number of Students: 3

Average Enrollment Date: 19-08-2023

Student: Nguyen, Eve - Enrollment Date: 19-08-2023, Email: eve.nguyen@dotnettutorials.net

Student: Wong, Alice - Enrollment Date: 15-08-2023, Email: alice.wong@dotnettutorials.net

Student: Zhang, Ivy - Enrollment Date: 23-08-2023, Email: ivy.zhang@dotnettutorials.net

Branch: Mechanical Engineering

Number of Students: 3

Average Enrollment Date: 20-08-2023

Student: Connor, Frank - Enrollment Date: 20-08-2023, Email: frank.oconnor@dotnettutorials.net

Student: Johnson, Bob - Enrollment Date: 16-08-2023, Email: bob.johnson@dotnettutorials.net

Student: Wilson, Jack - Enrollment Date: 24-08-2023, Email: jack.wilson@dotnettutorials.net

Branch: Electrical Engineering

Number of Students: 2

Average Enrollment Date: 19-08-2023

Student: Lee, Grace - Enrollment Date: 21-08-2023, Email: grace.lee@dotnettutorials.net

Student: Martinez, Carol - Enrollment Date: 17-08-2023, Email: carol.martinez@dotnettutorials.net

Branch: Civil Engineering

Number of Students: 2

Average Enrollment Date: 20-08-2023

Student: Kim, David - Enrollment Date: 18-08-2023, Email: david.kim@dotnettutorials.net

Student: Patel, Henry - Enrollment Date: 22-08-2023, Email: henry.patel@dotnettutorials.net

In the next article, I will discuss [How to Develop a Student Management Application using the EF Core Code First Approach](#) with Examples. In this article, I explain **LINQ to Entities in Entity Framework Core (EF Core)** with Multiple Examples. I hope you enjoy this LINQ to Entities in EF Core with Examples article.